

ChurnGuard

CUSTOMER CHURN PREDICTION — FINAL CAPSTONE REPORT

Combined Milestone 1 (Exploratory Data Analysis) and Milestone 2 (Model Building, Tuning, and Final Model Selection) into a single, end-to-end analysis and prediction pipeline.

Companion notebook: `Customer_Churn_Final_Report.ipynb` · August 2026

Contents

GETTING STARTED

1. Introduction

DIAGNOSIS

2. Data Cleaning and Pre-processing

DIAGNOSIS

3. EDA and Business Implication

PREDICTION & ACTION

4. Model Building and Interpretation

PREDICTION & ACTION

5. Model Improvement and Finalizing

PREDICTION & ACTION

6. Recommendations and Insights

WRAP-UP

7. Conclusion

All figures, tables, and statistics in this report are taken directly from the executed companion notebook (Customer_Churn_Final_Report.ipynb) — nothing here is re-estimated or restated from memory.

1. Introduction

ChurnGuard operates an e-commerce/subscription business where the account, not the individual customer, is the billing unit — a single account can carry several customers (Account_user_count). Across the 11,260 accounts analysed in this report, roughly one in six accounts — 16.8% — churns. That headline number is not, on its own, the real problem. The real problem is that today's retention process is reactive: a customer is contacted only after they have already cancelled, by which point the account, and every customer tagged to it, is typically already lost.

Two constraints make this worth solving now, rather than as a generic “reduce churn” exercise:

- **Cost asymmetry.** there is a cost asymmetry between acquiring a new account and retaining an existing one — any retention programme built on this analysis must be economically defensible, so a blanket, company-wide discount is not a viable response; it has to be targeted, and
- **No early-warning system exists today.** Customer Success and Retention teams currently have no way to score an active account's churn risk before that account leaves, so intervention only ever happens after the relationship is already over.

This report answers two connected questions in sequence. Sections 2–3 (“Diagnosis”) ask: where does churn actually concentrate, and why? Raw data is cleaned once, then examined through univariate, bivariate, and multivariate analysis to surface the handful of signals that genuinely separate accounts that leave from accounts that stay. Sections 4–6 (“Prediction and Action”) ask a harder question: can those same signals be combined into a single score precise enough to act on individual accounts — not just describe the problem in aggregate, but flag who is at risk, catch it early, and hand the business a short, targeted, defensible priority list. Every step of both halves is executed, end-to-end, in the companion notebook Customer_Churn_Final_Report.ipynb, which combines what were previously two separate milestone notebooks into one non-duplicative pipeline: data is cleaned exactly once, and every later analysis and model builds on that same cleaned dataframe.

2. Data Cleaning and Pre-processing

The raw export (11,260 rows, 19 columns, sourced from the “Data for DSBA” sheet of the Customer Churn Data workbook) carries three distinct data-quality issues. Each is identified and treated separately below, with the reasoning behind the chosen treatment made explicit — not just what was done, but why that option was preferred over the obvious alternatives.

2.1 Junk placeholder characters in numeric columns

Seven columns that should be purely numeric — Tenure, Account_user_count, rev_per_month, rev_growth_yoy, coupon_used_for_payment, Day_Since_CC_connect, and cashback — were instead stored as text (object) dtype, because a handful of stray junk characters (#, @, +, \$, *, and the literal string &&&&) were mixed in with genuine numeric entries; Login_device also carried &&&& as a corrupted category value. Left untreated, a column such as Tenure would be read by any downstream model as categorical text, silently discarding all of its numeric signal — including what Section 3 confirms is the single strongest churn driver in the dataset. The fix: every junk token is mapped to NaN, and the seven affected columns are then cast to numeric with pandas' `errors="coerce"`, so any value that still fails to parse as a clean number becomes a missing value rather than crashing the pipeline.

2.2 Duplicate category labels

Gender contained both “M”/“F” and “Male”/“Female” for the same two categories, and account_segment contained both “Regular +”/“Regular Plus” and “Super +”/“Super Plus” for the same two segments. Left unmerged, every later group-by would silently split one real category into two, understating its true size and its true churn rate. Both pairs are standardised to a single spelling before any further analysis.

2.3 Missing value treatment

After the two fixes above, almost every column carries some missing values — a mix of genuine blanks in the source export and the junk tokens just converted to NaN. None of the missingness rates are extreme (all comfortably below 10%, Figure 1), so rows are imputed rather than dropped — dropping would discard an otherwise-complete record just because one of its 19 fields is blank, silently biasing whichever accounts happen to remain.

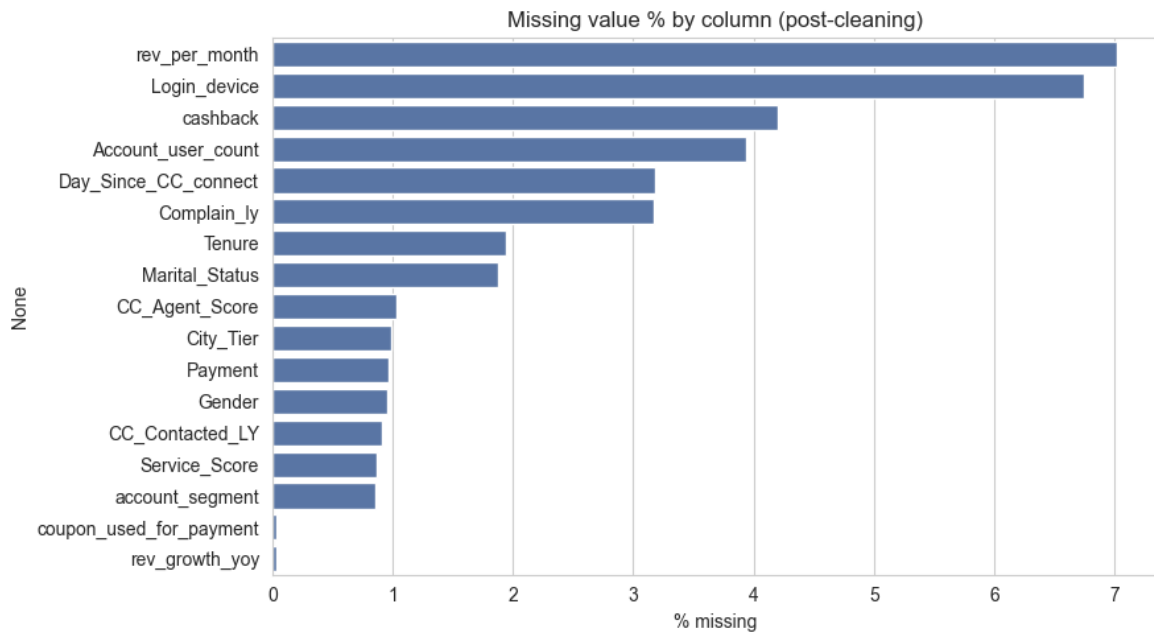


Figure 1. Missing-value percentage by column, after cleaning.

- **Continuous and count-like columns** → median. Most of these columns are right-skewed (confirmed in Section 3), and the median is robust to that skew, whereas the mean would be pulled toward the skewed tail and misrepresent a “typical” account.
- **Categorical columns** → mode (the most frequent category) — the standard, conservative choice when a category is missing at a low rate.

The target column, Churn, has zero missing values, so no target-side imputation is ever required.

2.4 Outlier detection and treatment

The boxplots in Figure 2 reveal two genuinely different patterns among the continuous columns, which call for two different treatments — this is the one deliberately judgement-based step in the whole cleaning process, and it is worth being explicit about both the pattern and the reasoning:

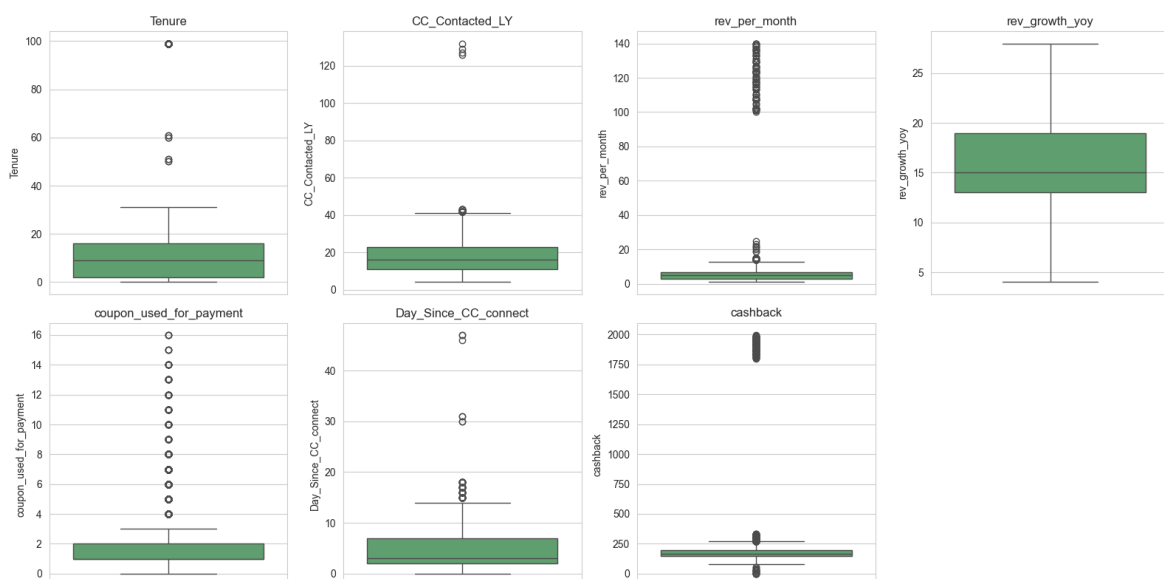


Figure 2. Boxplots of continuous variables before outlier treatment.

- **Tenure has a suspicious spike at exactly 99.** 131 accounts share the exact value 99, isolated from the rest of the distribution by a visible gap (the next-highest genuine values sit in the 50s–60s). That many accounts sharing one exact integer, with nothing in between, is the classic signature of a sentinel/placeholder value (e.g. “unknown” or “capped at export”), not 131 real 99-month-old accounts. Treatment: recode to missing, then re-impute with the median — the same honest treatment as any other missing value, not a fabricated number.
- **Four columns are genuinely, not artificially, skewed.** `rev_per_month`, `cashback`, `CC_Contacted_LY`, and `Day_Since_CC_connect` show real, if extreme, account behaviour (e.g. unusually high-value or highly-engaged accounts), not data errors. Treatment: winsorise (cap) at the 1st/99th percentile rather than delete the row — this keeps the account's other 18 fields intact while limiting how much leverage one extreme value can exert on an average or a model. The 1st/99th-percentile rule is a deliberately conservative middle ground: the stricter 1.5×IQR rule used to draw the boxplots above would flag far more points — as much as ~12% of some columns — as “abnormal”, over-correcting for what is more likely genuine skew than error. Percentile capping only adjusts the most extreme ~1–2% of values per column.

Column	Capped range	Values adjusted
<code>rev_per_month</code>	[1.0, 16.6]	113
<code>cashback</code>	[118.8, 329.3]	225
<code>CC_Contacted_LY</code>	[6.0, 40.0]	80
<code>Day_Since_CC_connect</code>	[0.0, 15.0]	93

Table 1. Winsorisation bounds and number of values capped per column.

2.5 Variables added

Two lightweight, business-interpretable features are added — no columns are removed at this stage, since every remaining raw field still carries some signal worth testing in Section 4:

- **`rev_per_user` = `rev_per_month` ÷ `Account_user_count`.** `rev_per_month` alone conflates “how much this account is worth” with “how many people are on it”; dividing by `Account_user_count` isolates the per-customer revenue contribution.
- **Tenure_Group.** a five-bucket bin of Tenure (0–3, 3–6, 6–12, 12–24, 24+ months). This does not add new information beyond the raw Tenure column — it repackages it for communication and for turning a risk pattern directly into a campaign rule (“target 0–3 month accounts”). The raw, continuous Tenure value, not this binned copy, is what is used inside the predictive model in Section 4, to avoid one signal competing with a redundant copy of itself.

3. EDA and Business Implication

This section works through univariate, bivariate, and multivariate analysis on the cleaned data, combining visual charts with the non-visual (tabular) evidence behind them, and closes each layer with what it means for the business — not just what the data shows.

3.1 Univariate analysis

The target variable is imbalanced: 83.2% of accounts are retained and 16.8% churn (Figure 3). Business implication: a model — or a manager — that assumes “most accounts are fine” is not wrong on average, but is useless for the specific task at hand, which is finding the one-in-six accounts worth intervening on. This is exactly why the modelling strategy in Section 4 evaluates on recall and F1 for the churn class rather than plain accuracy, and why class imbalance is explicitly handled (Section 4) rather than ignored.

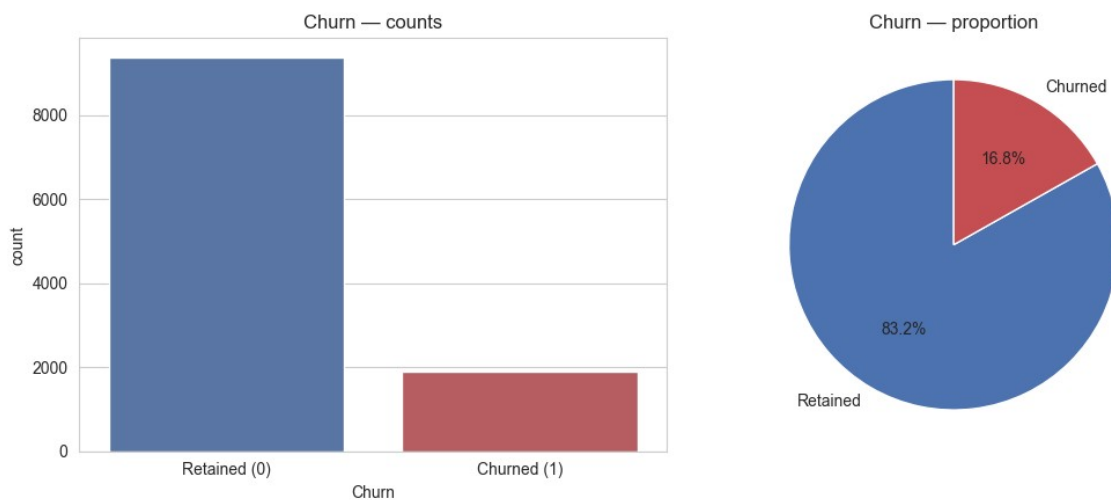


Figure 3. Churn distribution — counts and proportion.

Among the continuous variables (Figure 4), Tenure is heavily right-skewed — most accounts are young, with a long tail of loyal, long-tenured accounts. `coupon_used_for_payment` (skew ≈ 2.6) and the engineered `rev_per_user` (skew ≈ 3.5) are the most skewed of all: `coupon_used_for_payment` because most accounts use 0–2 coupons with a thin tail of heavy couponers, and `rev_per_user` because a small subset of accounts generates disproportionately high per-customer revenue. This confirms the median-imputation choice made in Section 2.3 was the right call for these columns.

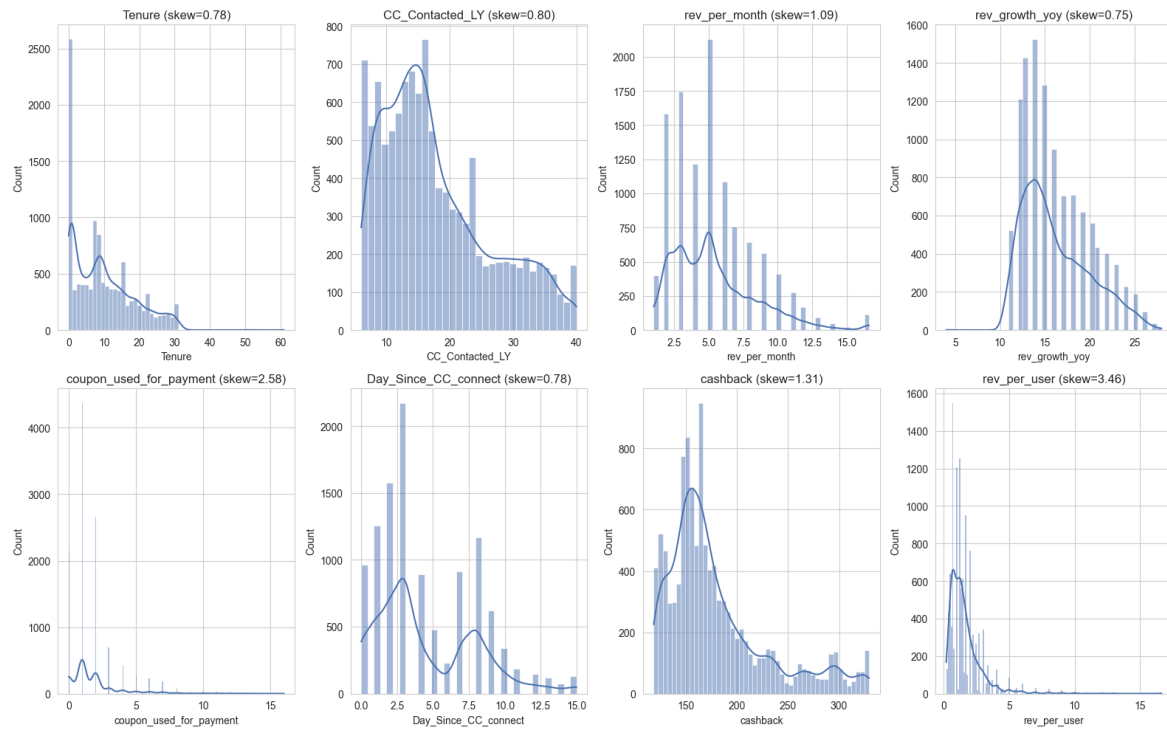


Figure 4. Distribution of continuous variables (with skew annotated).

Among the categorical fields (Figures 5–6), Debit/Credit Card dominate payment methods; Super and Regular Plus are the largest segments by volume; Married accounts are the largest marital-status group; Mobile dominates login device; and Tier-1 cities dominate. Roughly 28% of accounts logged a complaint in the last 12 months — a meaningful minority, and, as Section 3.2 shows, the strongest single categorical churn signal in the dataset.

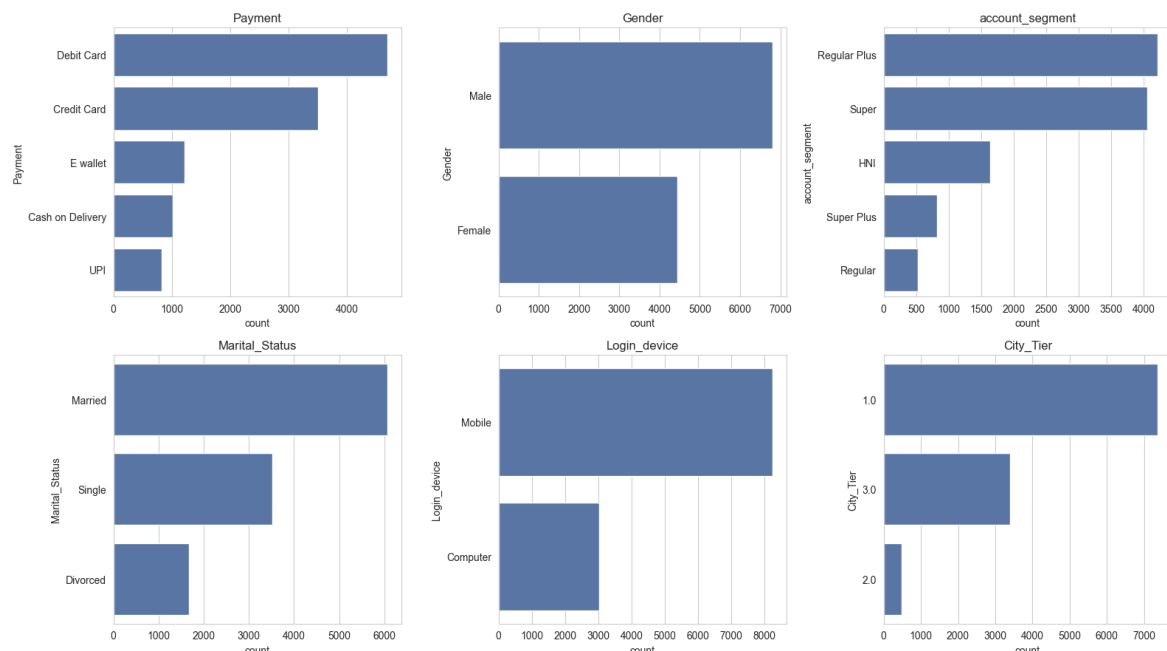


Figure 5. Distribution of categorical variables.

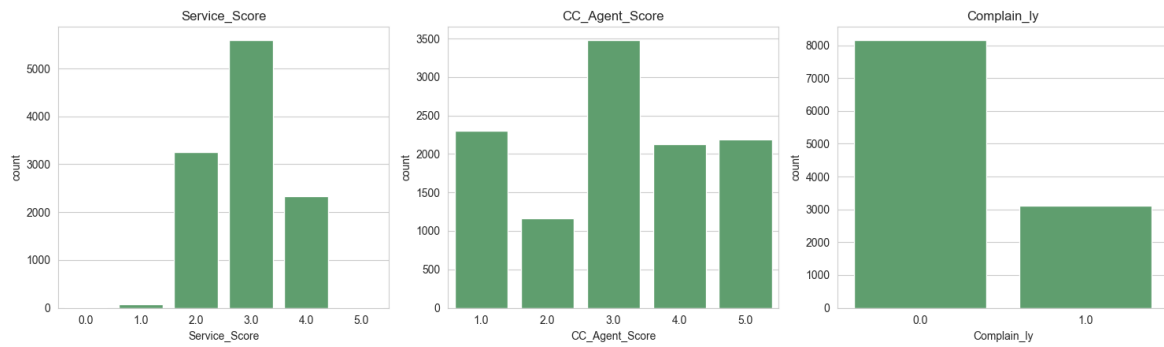


Figure 6. Distribution of Service_Score, CC_Agent_Score, and Complain_Iy.

3.2 Bivariate analysis

Churn rate by category (Figure 7, with the exact percentages behind it in Table 2) shows risk is not spread evenly across the customer base:

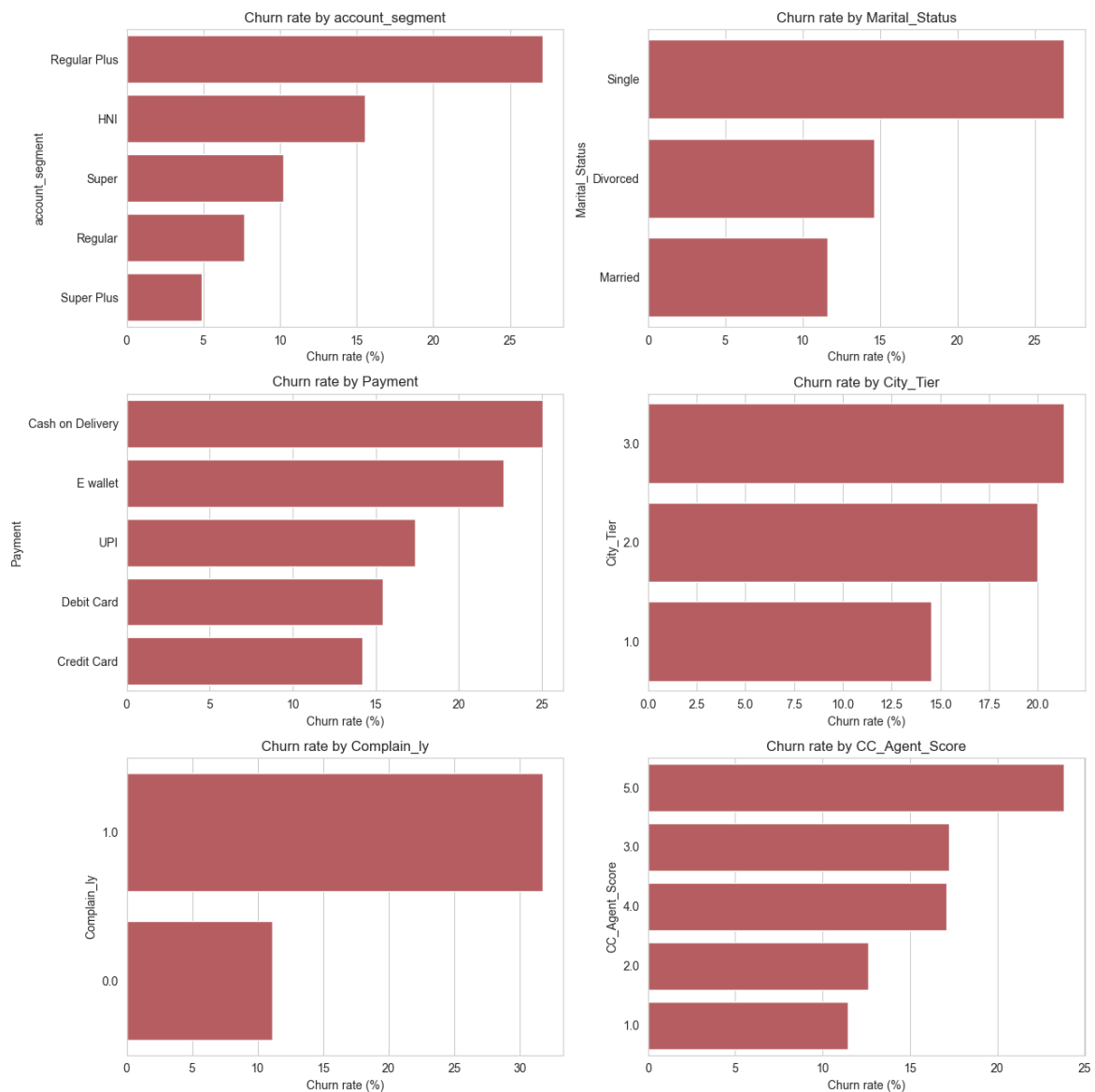


Figure 7. Churn rate by categorical variable.

Variable	Category	Churn rate
Complain_ly	Complaint logged	31.8%
Complain_ly	No complaint	11.1%
account_segment	Regular Plus (highest)	27.1%
account_segment	Super Plus (lowest)	4.9%
Marital_Status	Single	26.9%
Marital_Status	Married	11.6%
Tenure_Group	0–3 months	42.3%
Tenure_Group	24+ months	0.0%

Table 2. Churn rate by selected category, exact values from the cleaned data.

- **Complaints are the strongest categorical signal.** accounts with a logged complaint churn at 31.8%, nearly 3× the rate of accounts with no complaint (11.1%) — of every signal tested, this is the single most actionable trigger, because it is observable the moment it happens.
- **account_segment is non-monotonic.** Regular Plus churns most (27.1%) while Super Plus churns least (4.9%) — and Regular, a lower-spend segment, is actually one of the safer segments (7.7%). This is not simply “lower spend = higher risk”, so a blanket, spend-based targeting rule would miss the real pattern.
- **Marital status matters.** Single accounts churn at 26.9%, more than double the Married rate (11.6%).
- **Tenure_Group shows the clearest pattern of all.** risk is heavily concentrated in the first few months of an account's life — 42.3% in months 0–3, falling to essentially 0% beyond 24 months. This is the single clearest bivariate pattern in the whole dataset, and reframes churn as an onboarding-quality problem rather than a general loyalty problem.
- **CC_Agent_Score is counter-intuitive.** accounts giving the highest satisfaction score (5) churn more (23.8%) than those giving the lowest score (1, 11.5%) — plausibly because a top score is often logged right after an already-escalated complaint call, not a routine happy interaction. This feature is flagged for caution and should not be read naively (for example, as “agent training is failing”).

Against continuous variables (Figure 8, group means in Table 3), Tenure is again the clearest separator — churned accounts average 3.31 months versus 11.28 months for retained accounts. Day_Since_CC_connect is lower for churned accounts (3.35 vs. 4.80 days), i.e. churned accounts tend to have contacted customer care more recently before leaving — consistent with a “last straw” support interaction preceding departure.

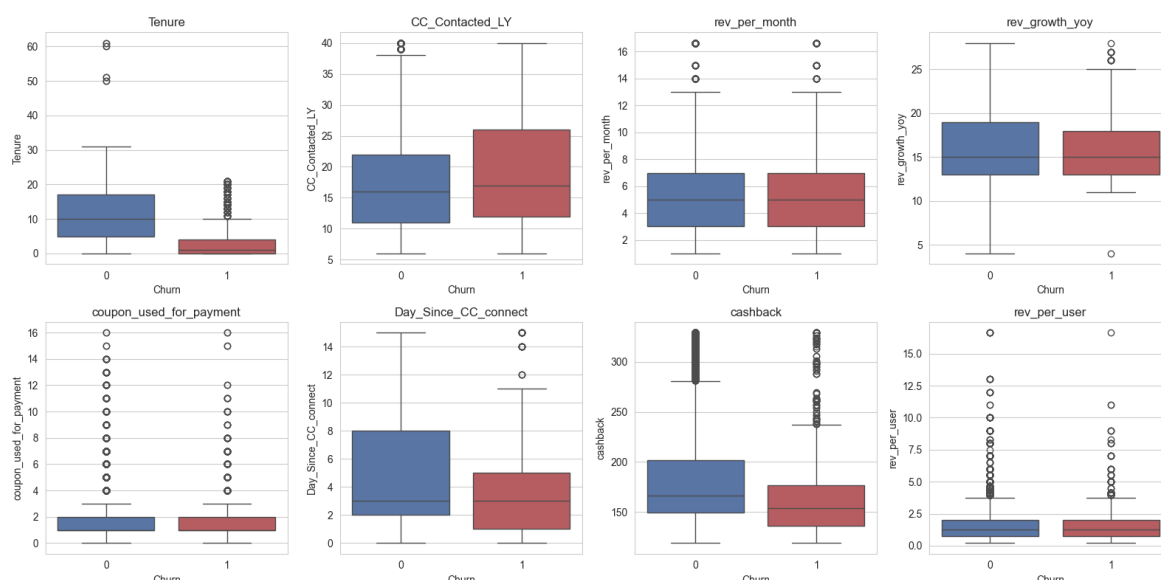


Figure 8. Continuous variables by churn status (boxplots).

Metric	Retained (Churn = 0)	Churned (Churn = 1)
Tenure (months)	11.28	3.31
Day_Since_CC_connect (days)	4.80	3.35
cashback	183.19	164.53
rev_per_month	5.24	5.57

Table 3. Group means for selected continuous variables, retained vs. churned accounts.

3.3 Multivariate analysis

The correlation heatmap (Figure 9) shows two off-diagonal values that stand out — `rev_per_month` vs. `rev_per_user` (+0.70) and `Account_user_count` vs. `rev_per_user` (−0.48) — but this is expected, not a red flag: `rev_per_user` is arithmetically derived from those two columns (Section 2.5), so some correlation with its own inputs is guaranteed by construction, not a coincidental data relationship. Excluding that one explained pair, the next-highest off-diagonal correlation among the remaining columns is a moderate 0.45 (`Tenure` vs. `cashback`) — well below the ~0.7–0.8 threshold conventionally treated as a serious multicollinearity concern, so no feature needs to be dropped for redundancy. Against the target, `Tenure` (−0.35) and `Complain_ly` (+0.25) are the two strongest linear signals — more than double the strength of any other numeric signal — consistent with the bivariate analysis above. All correlations against `Churn` are modest in absolute terms, which is expected: churn is a behavioural outcome driven by an interaction of factors, not any single variable acting alone. This is exactly why Section 4 tests tree-based ensemble models that can capture combinations, rather than relying on a linear read of one variable at a time — tree-based models are, in any case, robust to the `rev_per_month`/`Account_user_count`/`rev_per_user` collinearity noted above; it is flagged here only in the interest of full transparency.

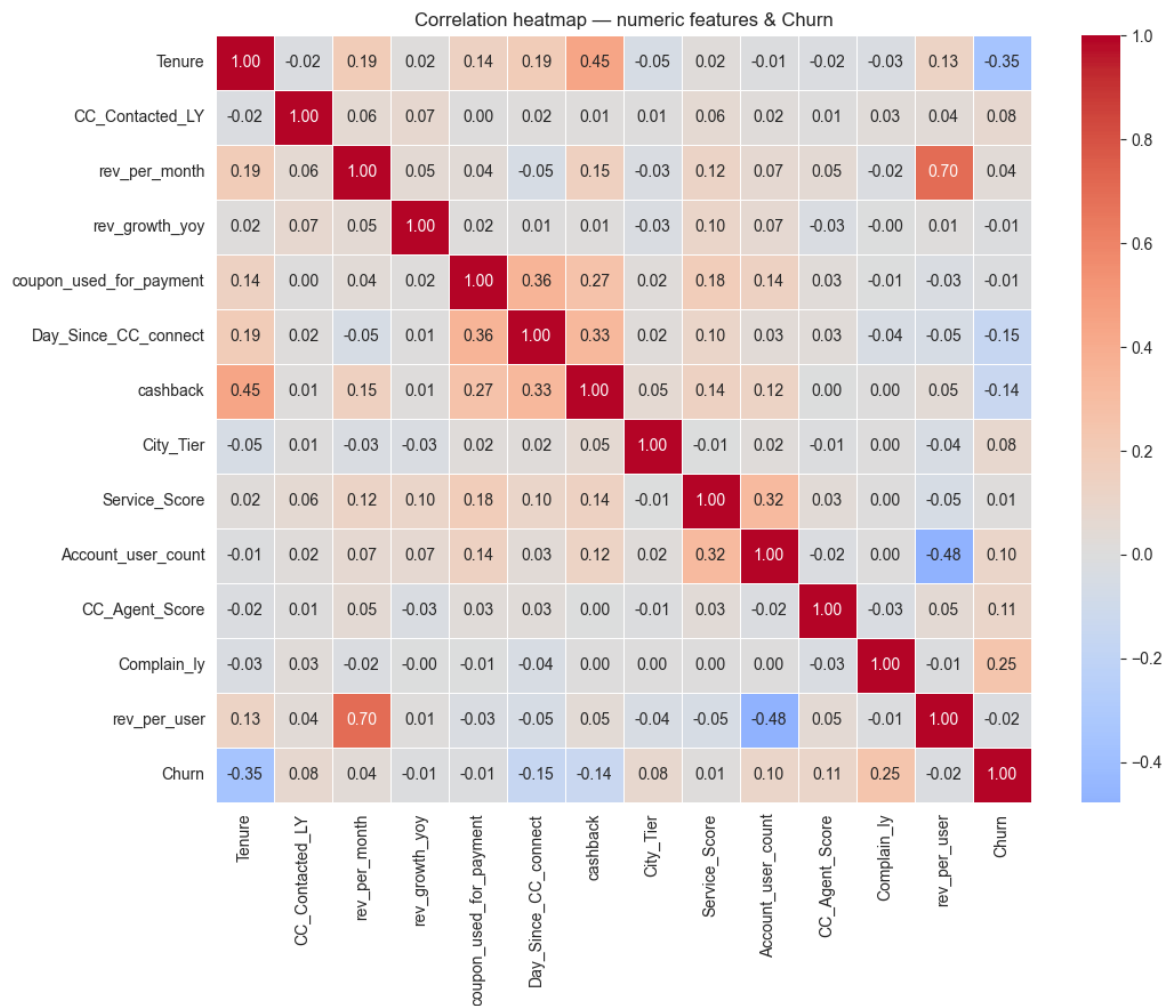


Figure 9. Correlation heatmap — numeric features and Churn.

3.4 Summary — business insights from EDA

- Churn is an onboarding problem, not a loyalty problem — risk is heavily concentrated in the first few months of an account's life.
- A logged complaint roughly triples churn risk — the single most actionable trigger identified.
- Regular Plus and Single account holders are disproportionately at-risk sub-populations — useful for segmenting a retention campaign rather than treating all accounts uniformly.
- Recent customer-care contact precedes churn — the window right after a support interaction is a critical moment for proactive outreach.
- The CC_Agent_Score finding needs a caveat attached before it informs any staffing decision.

4. Model Building and Interpretation

Before any features are encoded, two columns are deliberately excluded from the model: AccountID (a unique identifier, not a behavioural signal — including it would let a model memorise individual accounts instead of learning generalisable patterns) and Tenure_Group (a binned copy of Tenure, kept only for the charts in Section 3 — it carries strictly less information than the raw Tenure column it was derived from). The five remaining categorical columns are one-hot encoded (safe to do before splitting, since it never looks at the target); the data is then split 80/20, stratified on Churn so both the training and test sets preserve the same 16.8%/83.2% imbalance.

4.1 Rationale behind the choice of metric

With an 83.2%/16.8% split, a trivial model that predicts “retained” for every account already scores 83.2% accuracy while catching zero churners — it would look good on a scoreboard and be worthless to the business. Accuracy is therefore explicitly not the metric this project optimises for. Instead, three metrics are tracked for every model, chosen to match the actual business cost structure described in Section 1:

- **Recall (churn class).** of the accounts that actually churn, what fraction does the model catch? This is the priority metric: missing a real churner (a false negative) means losing that account with zero chance to intervene — the more expensive failure mode given the cost asymmetry in Section 1.
- **F1-score (churn class).** balances recall against precision, because recall alone can be gamed by a model that flags almost everyone as “at risk”, which would swamp the retention team with false alarms and burn budget on accounts that were never actually leaving.
- **ROC-AUC.** a threshold-independent view of overall separability, useful for comparing models before a specific risk-tiering cut-off is chosen in Section 5.

Precision is still reported for transparency, but is treated as a secondary check, not the metric models are selected on.

4.2 Rationale behind the choice of models

Five models are built, deliberately spanning a spread of complexity and inductive bias, not five arbitrary library picks:

Model	Why it's included
Logistic Regression	The transparent baseline every other model must beat.
K-Nearest Neighbours	A non-parametric baseline with no linear-separability assumption.
Gaussian Naive Bayes	Fast sanity check, though its feature-independence assumption is almost certainly violated (Section 3.3 showed features interact).

Model	Why it's included
Decision Tree	Captures non-linear splits directly; the interpretability precursor to the ensembles below.
Random Forest / XGBoost	Bagging and boosting ensembles — expected to be the strongest performers on this tabular, interaction-heavy dataset.

Table 4. Models built and the rationale for including each.

4.3 Cross-validated comparison

Every model is wrapped in the same pipeline shape — SMOTE, then (for the two distance/gradient-based models) a StandardScaler, then the classifier — and compared under identical stratified 5-fold cross-validation, so every model sees the same folds and every fold's SMOTE resampling touches only that fold's own training portion (never the validation fold), avoiding a subtle but common source of leaked, inflated CV scores.

Model	Recall	F1	ROC-AUC	Precision
KNN	0.944	0.856	0.983	0.783
Random Forest	0.848	0.883	0.988	0.921
XGBoost	0.846	0.879	0.985	0.914
Decision Tree	0.797	0.790	0.876	0.783
Naive Bayes	0.784	0.438	0.791	0.303
Logistic Regression	0.703	0.570	0.852	0.480

Table 5. 5-fold cross-validated mean scores, baseline (default hyperparameter) models.

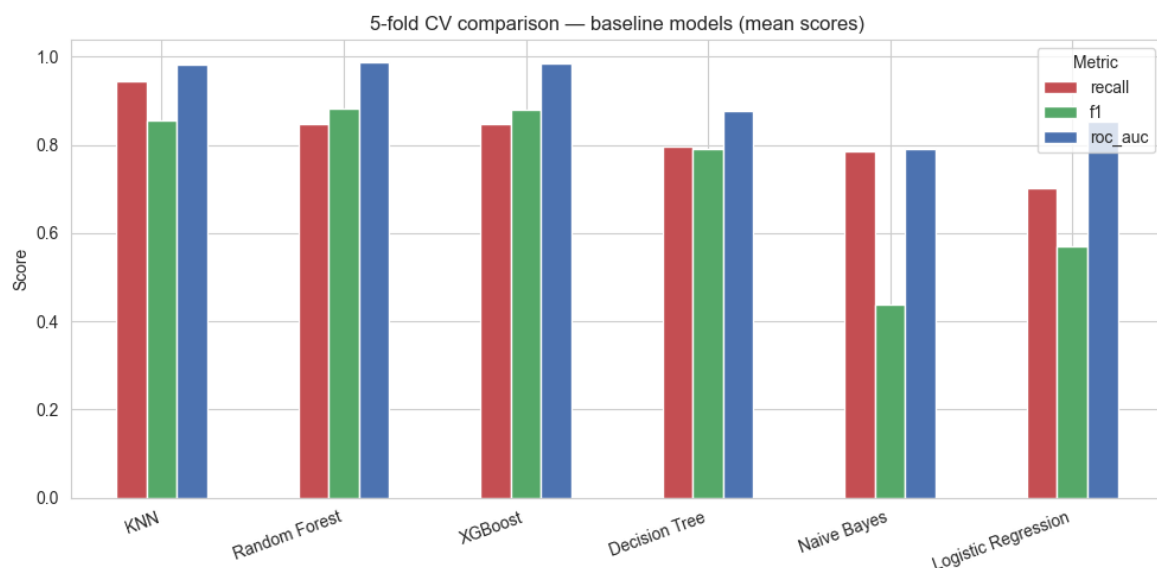


Figure 10. 5-fold CV comparison of baseline models across three metrics.

Naive Bayes and Logistic Regression trail on F1 — expected, since both assume the kind of simple (linear / feature-independent) structure that Section 3.3 already showed the real data

does not have. Naive Bayes's recall (0.784) looks fine in isolation, but its precision (0.303) is the worst of all six models — it reaches that recall mostly by over-flagging accounts, not by discriminating well.

KNN actually posts the single highest recall of all six models (0.944) — a genuinely honest result worth stating plainly rather than glossing over just because it is not the model this project ultimately favours. That recall comes at a real precision cost (0.783, second-lowest of the six), which is exactly the failure mode Section 4.1 flagged and the reason F1, not recall alone, is the metric hyperparameter tuning is optimised on in Section 5 — a model tuned purely for recall could win on that one number simply by flagging more accounts, without being more useful in practice.

Random Forest and XGBoost post the best F1 (0.883 / 0.879) and ROC-AUC (0.988 / 0.985) of all six models, consistent with the expectation in Section 4.2 that ensemble methods would lead on this tabular, interaction-heavy dataset — even though KNN edges them on recall alone, as noted above. These CV numbers reflect untuned, default-hyperparameter models; none of them are final. Section 5 tunes the strongest candidates and adds an explicit ensemble-of-tuned-models step before any model is selected for production use.

5. Model Improvement and Finalizing

5.1 Hyperparameter tuning — parameters searched and why

RandomizedSearchCV (30 draws, 5-fold stratified CV, the same SMOTE-in-pipeline design as Section 4.3) is used instead of an exhaustive GridSearchCV — the combined grids below have thousands of combinations, and a random sample of 30 captures most of the benefit at a fraction of the compute cost. Only Logistic Regression, Random Forest, and XGBoost are tuned: KNN and Naive Bayes were clearly weaker in Section 4.3 with no theoretical reason to expect tuning would close that gap, so tuning effort is concentrated on the candidates that can plausibly become the final model.

Tuning target: F1, not recall. Tuning directly on recall risks finding a setting that simply flags almost every account as “at risk” — F1 forces a genuine recall/precision trade-off. Recall, precision, and ROC-AUC are still all reported for the winning setting of each model, so nothing is hidden by optimising on a single number.

Model	Parameters searched	Why these, and this range
Logistic Regression	$C \in [0.01, 100]$ (log-spaced); class_weight $\in \{\text{None}, \text{balanced}\}$	C is the only real lever for a linear model — this spans heavily-regularised to almost-unregularised; class_weight is tested as an alternative/complement to the pipeline's own SMOTE.
Random Forest	n_estimators $\in [100, 600]$; max_depth $\in \{\text{None}, 5, 10, 15, 20\}$; min_samples_leaf $\in \{1, 2, 5, 10\}$; max_features $\in \{\text{sqrt}, \text{log2}\}$	max_depth / min_samples_leaf directly target the overfitting a single Decision Tree showed in Section 4.3; n_estimators trades stability for compute; max_features controls how de-correlated the trees are.
XGBoost	n_estimators $\in [100, 500]$; max_depth $\in \{3, 4, 5, 6\}$; learning_rate $\in [0.01, 0.3]$ (log-spaced); subsample & colsample_bytree $\in [0.6, 1.0]$	learning_rate and n_estimators are boosting's core bias/variance dial; max_depth bounds per-tree complexity; subsample/colsample_bytree inject randomness to fight boosting's known tendency to overfit if left unchecked.

Table 6. Hyperparameter search space and rationale, per tuned model.

Model	Best CV F1	Best parameters found
Logistic Regression	0.575	class_weight=None, C=0.01

Model	Best CV F1	Best parameters found
Random Forest	0.895	max_depth=20, max_features=log2, min_samples_leaf=1, n_estimators=437
XGBoost	0.897	colsample_bytree=0.85, learning_rate=0.10, max_depth=6, n_estimators=487, subsample=0.96

Table 7. Winning hyperparameters found by RandomizedSearchCV for each tuned model.

5.2 Honest comparison — tuned vs. baseline

Model	Stage	Recall	F1	ROC-AUC	Precision
Logistic Regression	Baseline	0.703	0.570	0.852	0.480
Logistic Regression	Tuned	0.710	0.575	0.856	0.483
Random Forest	Baseline	0.848	0.883	0.988	0.921
Random Forest	Tuned	0.860	0.895	0.990	0.933
XGBoost	Baseline	0.846	0.879	0.985	0.914
XGBoost	Tuned	0.867	0.897	0.989	0.931

Table 8. Baseline vs. tuned 5-fold CV mean scores.

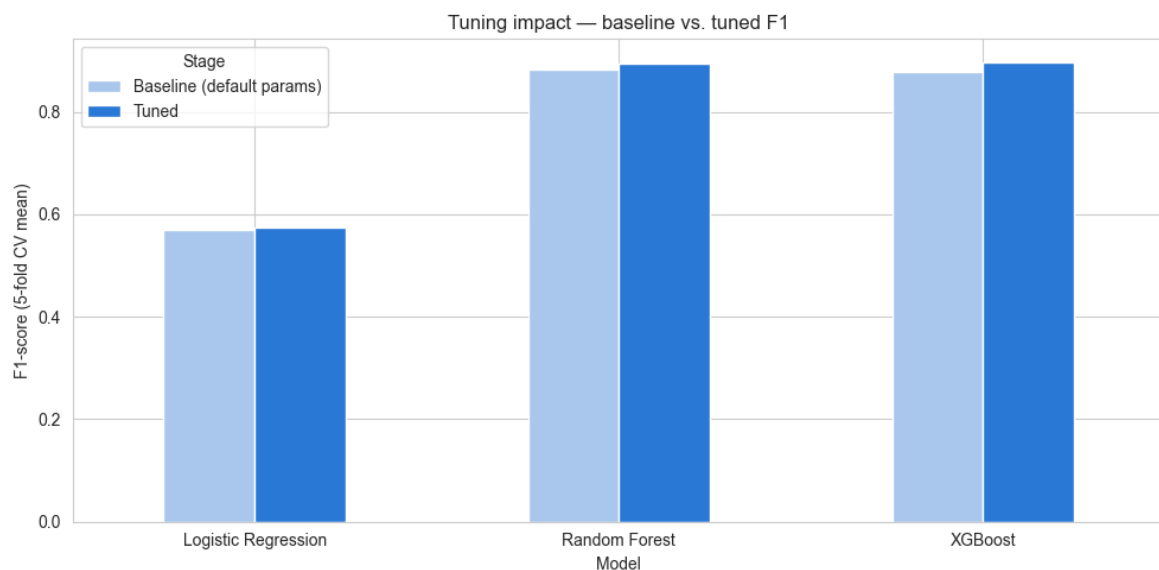


Figure 11. Tuning impact on F1-score — baseline vs. tuned.

Tuning delivers a real, if incremental, lift for Random Forest (F1 0.883 → 0.895) and XGBoost (F1 0.879 → 0.897) — consistent with both already being close to their ceiling at default settings. Logistic Regression's improvement is smaller still (F1 0.570 → 0.575), because no amount of hyperparameter search changes the fact that it is a linear model applied to a problem Section 3.3 already showed has non-linear, interacting structure — tuning a simple model harder does not turn it into a more complex one. This is reported as-is rather than overstated, in keeping with this report's standing rule of surfacing unflattering findings, not just favourable ones.

5.3 Ensemble technique — soft-voting the tuned models

As an explicit ensembling effort beyond Random Forest's and XGBoost's own internal bagging/boosting, a soft-voting VotingClassifier is built over all three tuned pipelines above. Soft voting averages each model's predicted churn probability (rather than a hard majority vote on the predicted class), letting a confident, correct model outweigh a less-confident one.

Model	Recall	F1	ROC-AUC	Precision
XGBoost (tuned)	0.867	0.897	0.989	0.931
Random Forest (tuned)	0.860	0.895	0.990	0.933
Voting Ensemble (all three tuned models)	0.844	0.870	0.982	0.897
Logistic Regression (tuned)	0.710	0.575	0.856	0.483

Table 9. Final comparison — tuned single models vs. the soft-voting ensemble.

Honest result: the voting ensemble did not win. Tuned XGBoost tops the table (F1 0.897), tuned Random Forest is essentially tied just behind it (F1 0.895), and the soft-voting ensemble of all three tuned models actually scores lower than either one alone (F1 0.870) — worse recall, worse ROC-AUC, and worse precision than both. The reason is visible in the table above: the ensemble's third member, tuned Logistic Regression, is far weaker than the other two (F1 0.575), and soft voting averages in its confidently-wrong probability estimates on every prediction, dragging the blended score down rather than adding useful diversity. This is reported as-found rather than forced into a “the ensemble won” narrative that the numbers do not support.

5.4 Final model selection

The model selected — ranked on F1, with recall and ROC-AUC checked alongside it — is the tuned XGBoost model on its own, not the ensemble. This also matches the expectation set in Section 4.2: XGBoost was flagged as the likely strongest tabular-data performer before any of this section's results existed.

5.5 Test-set evaluation

Every number up to this point came from cross-validation on the training set only. The table below and Figure 12 are the honest, held-out answer to “does this actually work on accounts the model never saw during training or tuning”:

Class	Precision	Recall	F1-score	Support
Retained	0.98	0.99	0.99	1,873
Churned	0.97	0.89	0.92	379
Accuracy			0.98	2,252

Table 10. Final XGBoost model — test-set classification report. Test ROC-AUC = 0.994.

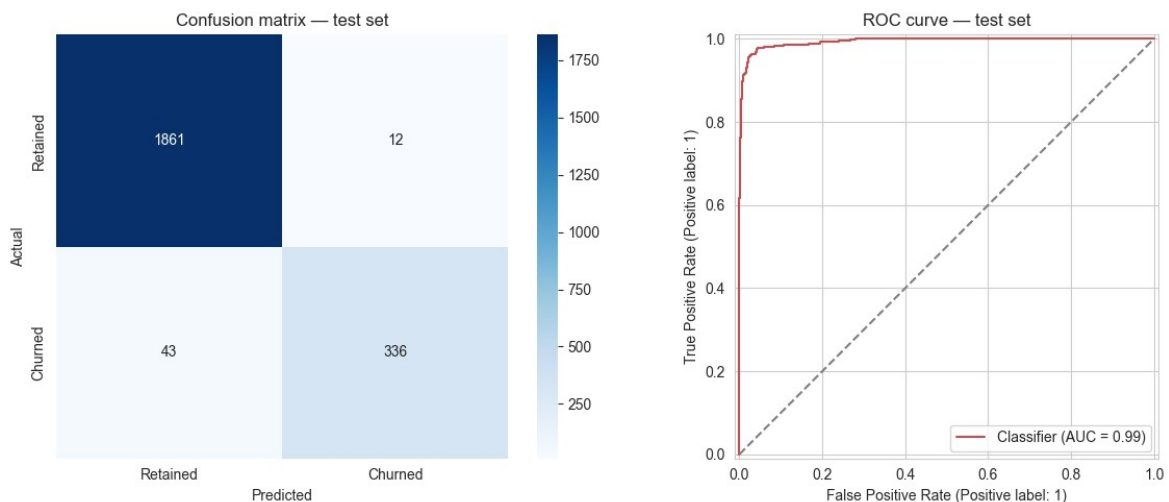


Figure 12. Confusion matrix and ROC curve — final model on the held-out test set.

On 2,252 held-out accounts never used for training or tuning, the final model recovers 89% of actual churners (recall) at 97% precision, with an overall test ROC-AUC of 0.994. The confusion matrix's false-negative cell — actual churners the model predicted as retained — is the single most business-costly cell in this whole report: every account in it is a real churner the model missed, which, given the cost asymmetry in Section 1, is a materially worse mistake than a false positive (an account wrongly flagged, costing at most a wasted retention offer).

5.6 Feature importance

Feature importance is read from the tuned XGBoost model directly (a bare LogisticRegression or VotingClassifier wrapper would not expose a single `.feature_importances_` of its own).

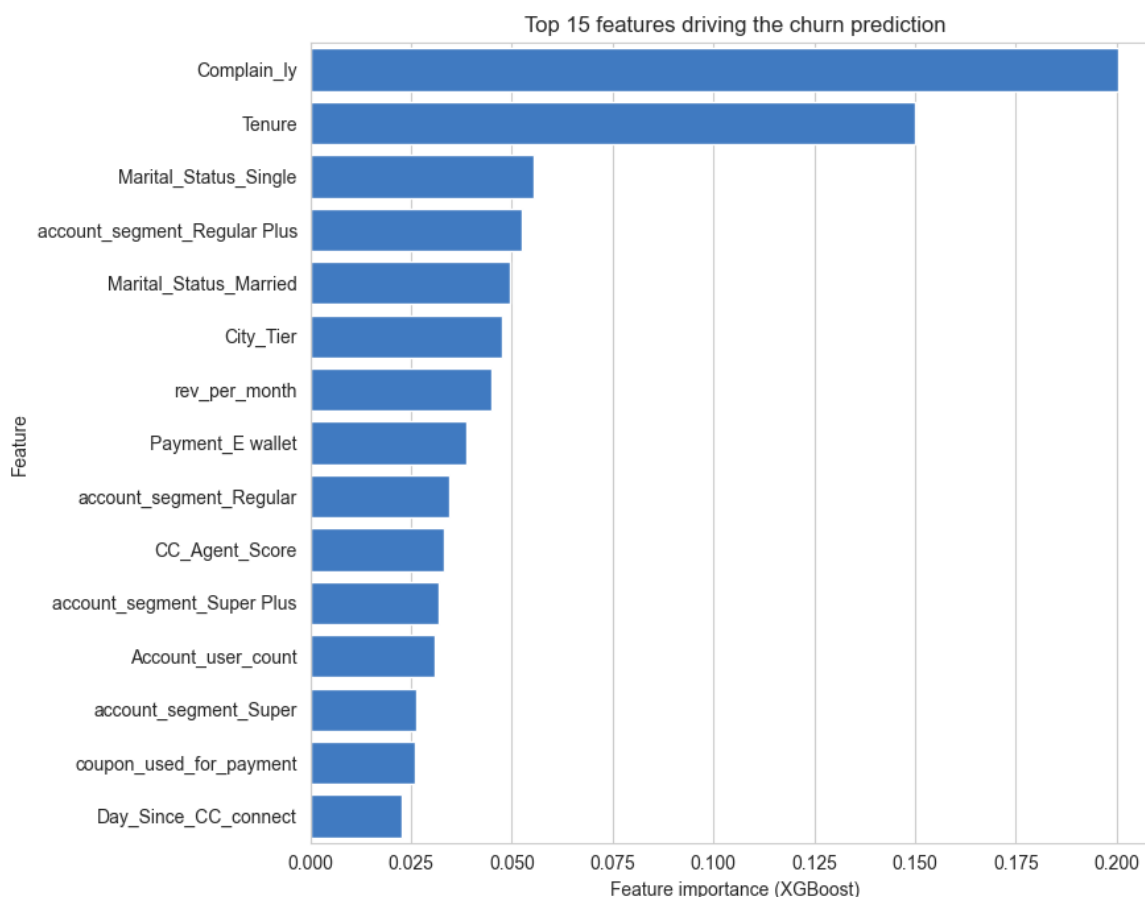


Figure 13. Top 15 features driving the churn prediction (XGBoost split-based importance).

Complain_ly and Tenure are, by a wide margin, the two most important features to the model — exactly the two signals flagged independently by the EDA in Section 3, via a completely different method (split-based importance from a boosted-tree model, versus group-by churn-rate differences in Section 3). Finding them again here is a genuine cross-check that the model has learned real business patterns, not spurious noise in the training data.

5.7 Risk tiering — turning a probability into an action

A raw probability such as “0.42” is not directly actionable for a retention team. It is converted into four plain risk tiers so each account gets a clear, consistent action:

Risk tier	Probability range	Accounts (test set)	Actual churn rate
Low	0.00 – 0.20	1,843	0.9%
Medium	0.20 – 0.50	61	44.3%
High	0.50 – 0.80	46	84.8%
Critical	0.80 – 1.00	302	98.3%

Table 11. Risk-tier calibration on the held-out test set.

The actual churn rate rises monotonically from Low (0.9%) through Critical (98.3%) — a well-calibrated score, and the direct bridge from “the model works” (Section 5.5) to “here's what the business does with it” (Section 6). The Critical tier is the short, defensible priority list Retention acts on first.

6. Recommendations and Insights

6.1 Meaningful, actionable insights

- **1. Churn is an onboarding problem, not a loyalty problem.** risk is heavily concentrated in the first few months of an account's life (42.3% in months 0–3 vs. ~0% past 24 months, Section 3.2) — this is fixable with a time-boxed intervention rather than an open-ended one.
- **2. A logged complaint roughly triples churn risk.** 31.8% vs. 11.1% (Section 3.2) — of every signal tested, this is the single most actionable trigger, because it is observable the moment it happens, not something that must be inferred after the fact.
- **3. Regular Plus and Single account holders are disproportionately at-risk.** Regular (lower spend) is actually one of the safer segments, so a blanket “target low-value accounts” policy would miss the real pattern (Section 3.2).
- **4. A model can now produce a ranked, actionable priority list.** every active account's churn probability can now be scored and sorted into Low/Medium/High/Critical tiers (Section 5.7), with the tiers empirically confirmed to carry a rising real-world churn rate — the concrete artefact that turns “we think churn is a problem” into “here is today's ranked list of accounts to call.”
- **5. The CC_Agent_Score finding needs a caveat.** a high satisfaction score correlating with higher churn is almost certainly measuring “logged right after an escalated complaint call”, not “agents are doing a bad job.” Acting on this signal without that context risks a wrong staffing decision (Section 3.2).

6.2 Detailed recommendations for management

- **1. Stand up a 0–3-month onboarding risk workflow.** route every newly created account through a lightweight automated check-in cadence (e.g. proactive contact at day 14 and day 45) rather than waiting for the standard churn score alone — the tenure effect is strong enough to justify a blanket rule for this specific window, on top of the general model.
- **2. Treat every logged complaint as an immediate retention trigger, not just a support ticket.** route complaint-flagged accounts to a same-week retention follow-up automatically — the cheapest, fastest-acting lever available from this analysis.
- **3. Prioritise outreach using the Critical risk tier.** refreshed on a regular scoring cadence (e.g. weekly) — this converts the model from a one-off analysis into an operational, repeated decision-support tool for the Retention team.
- **4. Pilot a segment-aware offer for Regular Plus and Single accounts.** rather than a single company-wide discount (Section 1) — this keeps the intervention cost down (a Finance/Revenue Assurance constraint) while targeting where the analysis shows the actual risk concentration sits.
- **5. Do not use CC_Agent_Score in any agent-performance or staffing decision without further, separate investigation.** flag this explicitly to whichever team owns customer-care KPIs, since the naive reading of this number points the wrong direction.
- **6. Re-validate this model periodically, not just once.** customer behaviour, pricing, and competitive pressure change over time; the tuning and evaluation in Section 5 reflect the account base as of this export, and the model's calibration (Section 5.7) should be re-checked whenever a materially new batch of accounts is added.

6.3 Recommendations for the client-facing / Customer Success team

- Use the risk tiers, not the raw probability number, when briefing frontline retention agents — a tier label (“Critical”) is faster to act on in a live conversation than a percentage.
- When contacting a Critical-tier account, lead with whichever top individual insight actually applies to that account (recent complaint, short tenure) — a generic retention script is weaker than one grounded in the account's own known risk driver.
- Track whether the false-positive accounts (flagged Critical but ultimately stayed) still respond positively to being proactively contacted — even a “wrong” flag that leads to a helpful check-in call is not wasted effort, but this should be measured, not assumed.

7. Conclusion

This report set out to answer two connected questions for ChurnGuard: where does churn actually come from, and can it be predicted precisely enough to act on individual accounts before they leave?

The diagnosis half (Sections 2–3) answered the first question with real, business-interpretable evidence: churn is not spread evenly across the customer base — it concentrates in new accounts, complaint-flagged accounts, and specific segments, and the two strongest signals, Tenure and Complain_ly, are each independently actionable the moment they are observed. The prediction half (Sections 4–6) answered the second question by building, honestly comparing, and tuning five models plus an explicit soft-voting ensemble, then converting the winning model's output into calibrated risk tiers rather than leaving it as an abstract probability. Every comparison in this report — baseline vs. tuned, single model vs. ensemble — was reported as it actually came out, including the finding that the ensemble did not outperform the best single tuned model, because a model that is trusted enough to act on has to survive exactly that kind of scrutiny.

The practical result is a short, defensible, continuously-refreshable priority list the Retention team can act on this week — a final XGBoost model that recovers 89% of actual churners at 97% precision on held-out data, with a test ROC-AUC of 0.994 — backed by a small number of clear, specific reasons why each account on it is at risk. That is a materially different, more useful outcome than the reactive, after-the-fact process ChurnGuard runs today.